CSE 211 — Theory of Computation

Introduction · Alphabets, Strings, Languages and Problems

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — the Q1(a) theory questions on ToC, strings and languages

Lesson 1 — Why Theory of Computation? — limits of machines, the memory hierarchy, three areas, hard problems

Lesson 2 — Alphabets, Strings, Languages and Problems — Σ, Σ*, ε vs ∅, set-former notation, problems as languages

Quick Notes — Introduction & Languages — revision bullets + key diagram

NOTE Slide numbers refer to the PDF pages of Lec01 - Introduction.pdf (53 pages) and Lec02 - Languages.pdf (52 pages). The slides are beamer builds, so one idea often spans several pages (e.g. sl. 8–11). The 'three areas of ToC' framing also appears in Atif Sir's Lecture 1 (sl. 2–3). Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Since 2019 every Section A starts with a short 'big-picture' theory part (4–15 marks). These two lectures answer them.

Session	Questions from these two lectures
2023-24	Q1(a) Options when a problem appears computationally hard (8)
2022-23	Q1(a) Public and private keys in asymmetric cryptography; the role of ToC (10)
2021-22	Q1(a) Password-protected login: which central area of ToC applies? (4)
2019-20	Q1(b) A scenario that specifically needs computationally hard problems (15)
2007-08	Q1(c) How set-formers are used to define a language (5⅓)
2006-07	Q1(c) φ and ε as languages over any alphabet — explain; are they equivalent? (5⅓)

EXAM FOCUS Four of the last four papers (2019-20, 2021-22, 2022-23, 2023-24) open with a ToC-in-the-real-world question worth 4–15 marks: hard problems, password systems, public-key cryptography, options for hard problems. Learn the three areas (automata · computability · complexity) and one concrete example of each, then the four model answers in Lesson 1. Lesson 2 (ε vs ∅, set-former notation) was asked only in 2006-07 and 2007-08, but its vocabulary is used in every other question.

Lesson 1 — Why Theory of Computation?

Lec01, slides 1–53 · three areas from Atif Sir Lecture 1, sl. 2–3

1. The Course Is About the Limits of All Machines (sl. 2–6)

Four problems look easy but lead to the whole course:

Problem	What it really asks	Model that settles it
Balanced parentheses (((…)))	Can finite memory track arbitrary nesting?	No → needs a stack (PDA, CFG)
Valid JSON	Remember what must still be closed: { [{ [	Stack-like memory → PDA
Program behaviour (while True: pass)	Will an arbitrary program halt?	Undecidable (halting problem)
Sudoku: check a filled board / solve an empty one	Is finding harder than checking?	P vs NP

Prediction game (sl. 5–6): for each problem ask two questions — *Solvable?* (computability) and *Efficient?* (complexity). Password rules: yes / yes. Balanced parentheses and JSON: yes / yes, but not with finite memory. Halting: no. Arbitrary Sudoku: yes, but no efficient algorithm is known. Perfectly verifying AI-generated code: no (it would decide halting).

2. Four Mysteries and the Hierarchy of Memory (sl. 7–24)

1. **Finite memory vs deep structure.** Password rules (≥ 8 characters, a digit, an uppercase letter, no spaces) need only finite memory → finite automata (sl. 8–11). A compiler's tokenizer turns `if (x == 10) return x + 1;` into IF LPAREN ID EQEQ NUM … using regular expressions, DFA and NFA (sl. 12–15). But finite memory cannot check arbitrary nesting such as ((…)) (sl. 16–19).
2. **Can every problem be solved algorithmically?** An editor that knows a JSON object is incomplete remembers the unclosed { [{ [— stack memory → pushdown automata (sl. 20–23).
3. **Can every program be analysed automatically?** No — see §3.
4. **Can every efficiently checkable solution be efficiently found?** The P vs NP question — see §4.

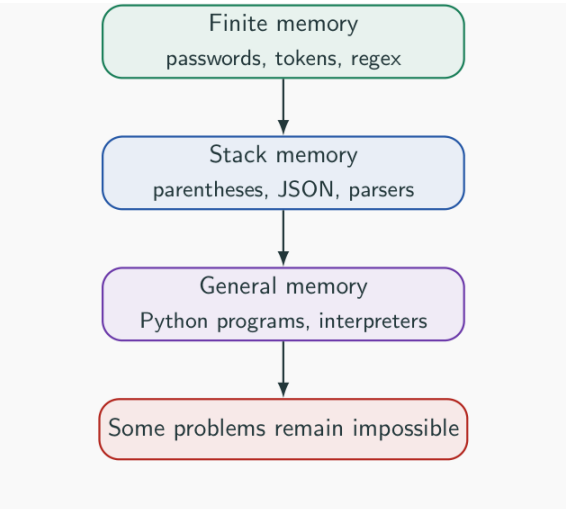


Figure 1.1 — The hierarchy of memory: finite → stack → general → impossible (sl. 24)

3. Will This Program Stop? (sl. 25–31)

print("Hello") obviously stops; while True: pass obviously does not. For an arbitrary program of thousands of lines no perfect halt-detector exists — 'this is not an engineering failure, it is a theorem' (sl. 28).

- **Halting problem:** no program can decide, for every program and input, whether it eventually halts (sl. 29).
- Consequences (sl. 30–31): no perfect bug detector, malware detector or program verifier for all programs; no AI can bypass this for arbitrary code. Useful tools exist; perfect tools do not.

4. Checking vs Finding — and AI (sl. 32–40)

Checking a completed Sudoku board is quick; finding a solution of an empty puzzle may need enormous search (sl. 32–34). This gap leads to P, NP, reductions and NP-completeness. An LLM may write a Sudoku solver, but writing a solver is not solving every instance instantly: AI improves heuristics, guides search and generates code, but **does not erase worst-case complexity** (sl. 35–37). The same question appears for theorem proving: can a system prove every theorem that has a short proof? (sl. 38–40) — proof verification, Lean, complexity.

5. The Three Areas and the Course Map (sl. 41–42; Atif Lec 1 sl. 2–3)

Area	Central question	Course topics	Everyday example
Automata theory	What can machines with limited memory do?	DFA, NFA, regex, CFG, PDA	Password validator, lexer, parser, JSON checker
Computability theory	What can be solved at all?	Turing machines, decidability, halting	No perfect bug/malware detector
Complexity theory	What can be solved efficiently?	P, NP, NP-complete, PSPACE	Sudoku, SAT, scheduling, cryptography

All three are linked by one question (Atif Lec 1 sl. 2): *what are the fundamental capabilities and limitations of computers?* Course map (sl. 41–42): DFA/NFA = finite memory; CFG/PDA = stack memory; Turing machines = computation itself; undecidability = impossible problems; complexity = feasible computation — 'the possible, the impossible, and the feasible'.

6. Where ToC Is Alive Today (sl. 43–53)

- AI: why LLMs generalise or hallucinate, their computational limits; AI for maths (Lean, formal verification).
- Software: SAT/SMT solvers, verification, compilers, static analysers, type systems, model checking.
- Complexity: P vs NP, approximation, hardness of optimisation, **cryptographic assumptions**; quantum computing, interactive and zero-knowledge proofs.
- Use AI as an assistant, not an oracle: ask for an automaton, then test it with counterexamples (sl. 43–45).

Exam Approach — ToC-in-the-World Questions

1. Name the area(s) first: automata (finite / stack memory), computability (solvable at all?), complexity (solvable efficiently?).
2. Tie the scenario to a concrete object: a regular language / DFA, an undecidable problem, or a hard (NP or harder) problem.
3. Say what is easy and what must be hard, and why (polynomial vs exponential).
4. One short example or number (e.g. 10^12 guesses for an 8-character password) makes the answer concrete.

Practice — Lesson 1

(a) Actual — 2021-22 Q1(a). (b) Actual — 2022-23 Q1(a). (c) Actual — 2019-20 Q1(b). (d) Actual — 2023-24 Q1(a).

(a) In the password-protected login system of a computer, which of the three traditionally central areas of the Theory of Computation has been applied? Explain clearly. (4)

(b) Elucidate the role of public and private keys in asymmetric cryptography. Assess how they work together to ensure secure communication between two parties. Analyse the role played by the field of Theory of Computation in this scenario. (10)

(c) Generally, an easy computational problem is preferable to a hard one. Describe one scenario which specifically requires computational problems that are hard, rather than easy ones. (15)

(d) What are the options that I have when I confront a problem that appears to be computationally hard? (8)

✓ Solution

(a) Two areas are at work.

- **Automata theory** — the password *rules* (length ≥ 8, contains a digit, an uppercase letter, no spaces) describe a regular language, so the check is done by a finite automaton / regular expression with finite memory (Lec01 sl. 8–11). Checking the typed password against the stored record is likewise a simple, efficient scan.
- **Complexity theory** — the *security* of the login. The system stores only a hash of the password. Computing the hash and comparing is easy (polynomial); recovering the password from the hash, or guessing it, should be infeasible: with 62 symbols and length 8 there are 62^8 ≈ 2.2 × 10^14 candidates, so brute force takes exponential time in the length. The system is secure exactly because that inversion problem is believed hard.
- Computability is not the issue: the attacker's problem is solvable (try all passwords) — just not efficiently.

(b) Each party has a key pair. The **public key** is published and used to *encrypt* messages to its owner (or to *verify* the owner's signature); the **private key** is kept secret and used to *decrypt* (or to *sign*). Secure communication: A obtains B's public key, encrypts the message with it, and sends the ciphertext; only B, holding the matching private key, can decrypt. For authenticity A also signs with A's private key and B verifies with A's public key. In RSA, the public key is (n, e) with n = p·q; the private key d satisfies e·d ≡ 1 (mod (p−1)(q−1)).

> **EXAM-READY ANSWER** Role of ToC (complexity theory): the scheme deliberately pairs an **easy** problem with a **hard** one. Encryption, decryption with the key, signing and verifying are all polynomial-time (in P). Computing the private key from the public key (e.g. factoring n) has no known polynomial-time algorithm — it is believed intractable. This is a one-way 'trapdoor' function: easy forward, hard backward unless you know the trapdoor (d). Breaking such schemes is in NP (a guessed factorisation or key is checked quickly), so if P = NP these systems would collapse; their security rests on hardness assumptions studied by complexity theory (Lec01 sl. 52: 'cryptographic assumptions').

(c) Cryptography and security. Here the designer wants a problem that is easy for the legitimate user but hard for an attacker:

- Passwords and hashing: computing a hash is easy, inverting it must be hard; otherwise anyone could recover passwords from a stolen database.
- Public-key encryption / digital signatures (RSA): multiplying two large primes is easy; factoring their product must be hard — the security of online banking and HTTPS depends on it.
- Proof-of-work in blockchains and CAPTCHAs use deliberately hard tasks to make abuse expensive.
- Why hardness is *required*: if the attacker's problem were in P, the attacker could solve it as fast as the user. Security therefore needs a gap — a function easy to compute but infeasible to invert

(super-polynomial time for every known algorithm). Theory of computation supplies the classification (P, NP, NP-hard) that lets us reason about that gap.

(d) Options (textbook, Sipser Ch. 7 discussion; slides Lec01 sl. 35–37):

1. **Make sure it is really hard** — try to reduce a known NP-complete problem to it; if you cannot, look for a polynomial algorithm (maybe it is in P).
2. **Solve a special case** — restrict the input (2-SAT instead of 3-SAT, trees instead of general graphs, small parameter k) where polynomial algorithms exist.
3. **Approximate** — accept a near-optimal answer with a guaranteed ratio (e.g. 2-approximation for vertex cover).
4. **Heuristics and practical solvers** — local search, SAT/SMT solvers, branch-and-bound, AI-guided search; often fast in practice though not in the worst case ('AI can guide search… but does not erase worst-case complexity', sl. 37).
5. **Exact exponential algorithms for small inputs** — backtracking, dynamic programming, when n is small.
6. **Randomisation / average case** — algorithms that work with high probability or on typical inputs.
7. **Change the problem** — relax constraints, or settle for checking a proposed solution (verification is easy).

Lesson 2 — Alphabets, Strings, Languages and Problems

Lec02, slides 1–52 · ε/∅ table from Lec03 sl. 19 and Lec05 sl. 83

1. Everything Is Strings (sl. 2–7)

42, 01101010, if (x > 5), ATTGCCGA, an emoji, 'Write a poem.' — all are finite sequences of symbols. Integers, DNA, programs, images (JPEG/PNG bit streams), audio and prompts are all encoded as strings: Data → Strings.

2. Alphabet, String, Length (sl. 8–12)

- **Alphabet Σ** — a finite, nonempty set of symbols: Σ1 = {0, 1}, Σ2 = {a, …, z}, Σ3 = {A, C, G, T}.
- **String over Σ** — a finite sequence of symbols of Σ: ε, 0, 1, 01, 1011, 111000.
- **Length** |w| — number of symbols: |1011| = 4, |ε| = 0. ε is the **empty string**.

3. String Operations (sl. 13–16)

```
Concatenation   01 · 10 = 0110          (εw = wε = w)
Power           0^2 = 00,  (ab)^3 = abababb,  w^0 = ε
Reverse         (10110)^R = 01101
Number of strings of length k over Σ:  |Σ|^k     (e.g. 2^3 = 8 binary strings of length 3)
```

4. Σ* and Σ+ (sl. 17–19; Lec03 sl. 25)

Σ* is the set of all finite strings over Σ: for Σ = {0, 1}, Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, …}. Σ* is infinite, but every string in it is finite. Σ+ = Σ* − {ε} (one or more symbols).

5. Languages (sl. 20–22)

A **language** is a set of strings, i.e. any subset L ⊆ Σ. *Over Σ = {0, 1}: L1 = {0, 10, 111} (finite), L2 = {w ∈ Σ : w contains an even number of 1s}* (infinite), L3 = Σ*, L4 = ∅. A language separates YES strings from NO strings.

Notation	Meaning	Number of strings
ε	the empty string (a string, length 0)	— (it is a string, not a set)
∅	the empty language	0
{ε}	the language containing only the empty string	1

TRAP ∅ ≠ {ε}. Concatenating with ∅ kills everything (L∅ = ∅), concatenating with {ε} changes nothing (L{ε} = L), and ∅* = {ε} (Lec05 sl. 12–13, 83).

6. Describing Languages: Set-Former Notation (textbook)

A language is usually written in **set-former (set-builder) notation** {w | property of w}: the part before the bar names a generic string, the part after the bar is a condition it must satisfy. Examples:

```
{ w ∈ {0,1}* : w contains an even number of 1s }
{ 0^n 1^n | n ≥ 0 }            = {ε, 01, 0011, 000111, …}
{ w ∈ {0,1}* : w = w^R }        (palindromes, sl. 31–34)
{ x : x is a syntactically valid JSON string }   (sl. 35–38)
```

Set-formers let us define infinite languages precisely and finitely, using parameters (n ≥ 0), quantifiers and string operations (reverse, length, counts such as #0(w)). Listing works only for finite languages.

7. Computational Problems Are Languages (sl. 23–46)

Decision problems — Is 100101 divisible by 3? Is (()()) balanced? Is this Python program syntactically valid? Is this Boolean formula satisfiable? — have YES/NO answers. Encode each input as a string and collect the YES instances: **a computational problem becomes a language** (sl. 26–27). Examples: L_even = {w : even number of 1s} (a tiny machine will recognise it), PAL (needs more memory than a finite automaton), JSON, PYTHON, SAT = {φ : φ is satisfiable} (YES: (x ∨ y) ∧ (¬x ∨ z); NO: x ∧ ¬x).

8. Which Machines Recognise Which Languages? (sl. 47–52)

Machine model	Memory idea	Language class
Finite automata	finite memory	Regular languages
Pushdown automata	one stack	Context-free languages
Turing machines	unbounded memory	Computable (decidable / RE) languages
Efficient algorithms	bounded resources	Complexity classes (P, NP, PSPACE)

Exam Approach

1. For 'define / explain' questions give the formal definition, a two-line example, and one contrast (ε vs ∅ vs {ε}, finite vs infinite language).
2. When asked to define a language, use set-former notation with an explicit alphabet and parameter range.
3. Count strings with |Σ|^k; order strings by length, then lexicographically (shortlex).

Practice — Lesson 2

(a) Actual — 2006-07 Q1(c). (b) Actual — 2007-08 Q1(c). (c)–(d) Constructed (from the slides' mini assignment, sl. 51).

(a) "φ, ε are languages over any alphabet" — explain. Can these two be called equivalent? Why? (5⅓)

(b) Explain how set-formers can be used to define a language. (5⅓)

(c) Over Σ = {0, 1}: how many strings have length exactly 4? length at most 4? List the first eight strings of Σ* in shortlex order.

(d) Define as languages: binary strings representing numbers divisible by 5; valid BUET student IDs of the form YYDDRRR (7 digits).

✓ Solution

(a) Strictly, ∅ (φ) is a language and ε is a string; the language meant is {ε}. ∅ is the language with no strings; {ε} is the language with exactly one string, the empty string. Both are languages over every alphabet, since ∅ ⊆ Σ* and ε ∈ Σ* for every Σ. They are **not equivalent**: |∅| = 0 but |{ε}| = 1, and they behave differently under the operations — L∅ = ∅ while L{ε} = L; ∅* = {ε} = {ε}*. A DFA for ∅ has no accepting reachable state; a DFA for {ε} accepts in its start state and rejects every non-empty string.

(b) See §6. A set-former {w | P(w)} defines the language as all strings w (over a stated alphabet) for which the predicate P holds, e.g. {w ∈ {a,b}* | w has equal numbers of a's and b's}, {0^n1^n | n ≥ 0}. It gives a finite, unambiguous description of an infinite set, and the membership test 'is w in L?' is exactly the decision problem the machine must solve.

(c) Length 4: 2^4 = 16. Length ≤ 4: 1 + 2 + 4 + 8 + 16 = 31. Shortlex: ε, 0, 1, 00, 01, 10, 11, 000.

(d) L5 = {w ∈ {0,1}+ | the number with binary representation w is divisible by 5}. L_ID = {d1d2…d7 ∈ {0,…,9}^7 | d1d2 = admission year, d3d4 = department code, d5d6d7 = roll}. Both are regular — the first by a 5-state remainder DFA (Lesson 4).

Quick Notes — Introduction & Languages

Revision points for Lessons 1–2.

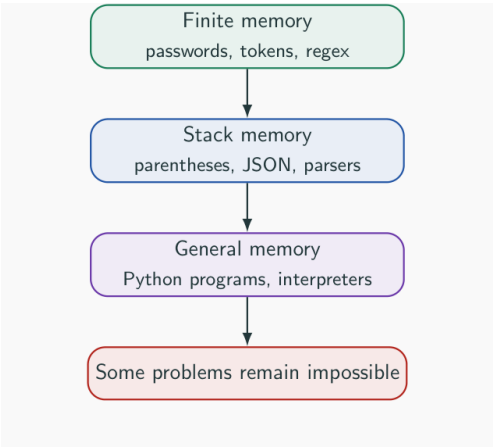
L1 · Why Theory of Computation

- Four examples: parentheses (stack), JSON (stack), halting (undecidable), Sudoku (P vs NP).
- Memory hierarchy: finite (passwords, tokens, regex) → stack (parentheses, JSON, parsers) → general (programs) → impossible.
- Three areas: automata (limited memory) · computability (solvable?) · complexity (efficiently solvable?).
- Halting problem: no perfect halt / bug / malware detector — a theorem, not an engineering failure.
- AI improves heuristics but does not erase worst-case complexity.
- Hard problems are needed for security: hashing, RSA (factoring), signatures, proof-of-work.
- Options for hard problems: confirm hardness, special cases, approximation, heuristics/solvers, small-n exact, randomisation.

L2 · Strings and Languages

- Alphabet = finite nonempty set; string = finite sequence; $|\varepsilon| = 0$; Σ^* infinite, $\Sigma^+ = \Sigma^* - \{\varepsilon\}$.
- Language = subset of Σ^*. *ε (string) ≠ $\emptyset$ (0 strings) ≠ $\{\varepsilon\}$ (1 string).* $L\emptyset = \emptyset$, $L\{\varepsilon\} = L$, $\emptyset = \{\varepsilon\}$.
- Set-former $\{w \mid P(w)\}$ defines infinite languages finitely.
- Problem = language of YES-instances. FA ↔ regular, PDA ↔ CFL, TM ↔ computable, bounded resources ↔ complexity classes.

Key Diagrams to Reproduce



Hierarchy of memory (sl. 24) — same as Fig. 1.1